Documents Notebook

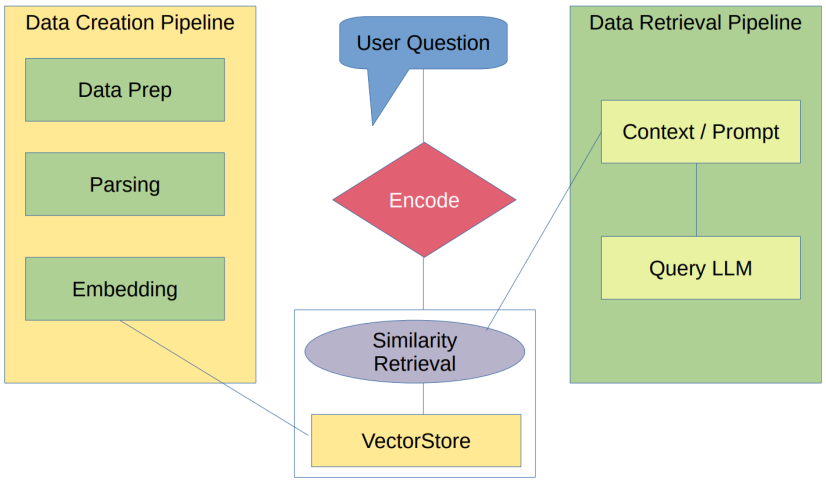
For this chatbot I will be using bash man pages, tealdeer output and the output from the bash help system for builtin commands.

There will need to be 2 pipelines.

- The first will be the data ingestion pipeline, which is in this notebook.
- Then we will need a query pipeline that will use our domain specific emnbeddings as the context for an LLM prompt. This is the RAG implementation that will be built. It is located in bashbot.ipynb and the associated script.

The dependencies are in requirements.txt.

Architecture



Data Ingestion Pipeline for Bashbot

Imports

```
import os
import sys
import shutil
from pathlib import Path
from langchain_core.documents import Document
from langchain_community.document_loaders import DirectoryLoader, TextLoader #,PyPDFLoader,
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

Function Definitions

load the corpus from the text files in the data directory

```
def load_txt_corpus():
    """ Loads all text files from the ./data directory."""
    loader = DirectoryLoader(
        "../data",
        glob="**/*.txt",
        loader_cls=TextLoader,
        loader_kwargs={'encoding': 'utf-8'},
        show_progress=True
    )
    corpus = loader.load()

    return corpus
```

```
def process_text_files(path):
    """Processes all text (.txt) files in the provided path"""
    documents = []
    file_dir = Path(path)

    files = list(file_dir.glob("**/*.txt"))
    print(f"Preparing to process {len(files)} text files...")

    # process the files and modify the source metadata to include manpage reference if poss
    for f in files:
#        print(f"processing {f.name}")
        try:
            loader = TextLoader(str(f))
            docs = loader.load()

            for d in docs:
                d.metadata['manpage'] = Path(f).stem
                d.metadata['source_file'] = f.name
                d.metadata['file_type'] = 'txt'
                d.metadata['platform'] = 'linux'
                d.metadata['source'] = 'manpage'

                documents.extend(docs)

        except Exception as e:
            print(f"    ERROR {e}")

    print(f"Total documents loaded: {len(documents)}.")
    print(type(documents))
    return list(documents)
```

```python
def process_all_files():
    """Wrapper function to allow calling functions to process
    multiple file types. Plan to integrate Docling to allow
    a multitude of files types in the future."""
    documents = []
    documents.extend(process_text_files("../data"))

    return documents
```

divide text into chunks

```python
# Splits the douments into manageable sized chunks.
def split_documents(corpus, chunk_size=500, chunk_overlap=200):
    """Chunks the text into chunk_size with an overlap specified"""
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        length_function=len,
        separators=["\n\n", "\n", " ", ""]
    )
    split_docs = text_splitter.split_documents(corpus)
    print(f"Splitting {len(corpus)} documents into {len(split_docs)} chunks.")
    if split_docs:
        print("\nSample chunk:")
        print(f"Page_content: {split_docs[0].page_content[:300]}...")
        print(f"Metadata: {split_docs[0].metadata}")

    return split_docs
```

Notebook Exploration

NOTE

If you want to use the existing vector db and skip parsing the docs, set the **USE_EXISTING=True** below

```python
USE_EXISTING=False
INIT=True
VERBOSE = False
```

To remove the exiting data store set INIT=True above.

Chromadb is thread-safe but not process safe. The method call in the VextorStore class will generate an error if used within the same process. To avoid this, you can clean the existing vectorstore by manually removing the ../data/vector_store directory or by setting INIT=True above.

```python
if INIT and not USE_EXISTING:
    store_path = Path("../data/vector_store/")
    print(f"Removing vector store at {store_path}.")
    shutil.rmtree(store_path, ignore_errors=True)
```

```
Removing vector store at ../data/vector_store.
```

Read data from files, and maybe an RDB

Embedding Manager

I want the flexibility to have pluggable vector store implementations. I am going to wrap this part in classes. Inspiration for this came from Krish Naik's YouTube video https://youtu.be/o126p1QN_RI?si=7xC5H47A3iuu52RK and pixegami https://youtu.be/2TJxpyO3ei4?si=zPNEsAFWWy5Cenzq

```python
import numpy as np
from sentence_transformers import SentenceTransformer
import chromadb
from chromadb.config import Settings
from chromadb.utils.batch_utils import create_batches
import uuid
from typing import List, Dict, Any, Tuple
from sklearn.metrics.pairwise import cosine_similarity

class EmbeddingMgr:
    """This class manages the embedding implementation.
    For now I am going to use a SentenceTransformer of
    some kind."""

    def __init__(self, model_name: str = "all-MiniLM-L6-v2", verbose: bool = False):
        """
        ctor

        Args:
            model_name: model name for sentence embeddings.

        Notes: For now I am using HuggingFace models.
        """

        self.model_name = model_name
        self.model = None
        self.verbose = verbose
        self._load_model()


    def _load_model(self):
        """This method loads the model defined in the ctor."""
```

```python
        try:
            print(f"Loading model {self.model_name}.")
            self.model = SentenceTransformer(self.model_name)
            print(f"Model loaded successfully.")
            print(f"Embedding dimension: {self.model.get_sentence_embedding_dimension()}")
        except Exception as e:
            print(f"Exception while loading model {self.model_name}: {e}")
            raise

    def generate_embeddings(self, texts: List[str]) -> np.ndarray:
        """
        Generate embeddings for a list of text strings

        Args:
            texts: List of strings

        Returns:
            numpy array of embeddings with shape (len(texts), embedding_dim)
        """
        if not self.model:
            raise ValueError("Model has not been loaded.")

        if VERBOSE:
            print(f"Generating embeddings for {len(texts)} strings.")
        embeddings = self.model.encode(texts, show_progress_bar=self.verbose)
        if VERBOSE:
            print(f"Generated embeddings with shape: {embeddings.shape}")
        return embeddings
```

Initialize the Embedding Manager

```python
emb_mgr = EmbeddingMgr("all-MiniLM-L6-v2", verbose=False)
emb_mgr
```

```
Loading model all-MiniLM-L6-v2.
```


```
{"model_id":"11e6f54b3d5f4268979a132bf06c4b08","version_major":2,"version_minor":0}
```


```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you ex

Model loaded successfully.
Embedding dimension: 384
```

```
<__main__.EmbeddingMgr at 0x7fa3046b04a0>
```

VectorSrore Database

```python
class VectorStore:
    """Manages embeddings in a vector database"""

    def __init__(self, collection_name: str = "bashdocs", persist_directory: str = "../data/
        """
        ctor

        Args:
            collection_name: Name of the collection
            persist_directory: Path to directory for persistant vectors.
        """
        self.collection_name = collection_name
        self.persist_directory = persist_directory
        self.client = None
        self.collection = None
        self._initialize_store()

    def _initialize_store(self):
        """Initialize the ChromaDB client and collection"""
        try:
            # Create the persistent ChromaDB client
            os.makedirs(self.persist_directory, exist_ok=True)
            self.client = chromadb.PersistentClient(path=self.persist_directory)

            # Get or create the collection
            self.collection = self.client.get_or_create_collection(
                name=self.collection_name,
                metadata={"description": "Bash commands document embeddings for RAG"}
            )
            print(f"Vector store insitialized for collection {self.collection_name}")
            print(f"Exisiting documents in collection: {self.collection.count()}")

        except Exception as e:
            print(f"Error initializing vector store: {e}")
            raise

    def add_documents(self, documents: List[Any], embeddings: np.ndarray):
        """
        Add documents to the vector store

        Args:
            documents: List of langchain Documents
```

```
    embeddings: The embeddings for the documents
"""
if len(documents) != len(embeddings):
    raise ValueError("Number of documents to embeddings mismatch.")

print(f"Adding {len(documents)} documents to the vector store...")

# Prepare data for ChromaDB
ids = []
metadatas = []
documents_text = []
embeddings_list = []

for idx, (doc, embedding) in enumerate(zip(documents, embeddings)):
    # Generate the uuid
    doc_id = f"doc_{uuid.uuid4().hex[:8]}_{idx}"
    ids.append(doc_id)

    # Metadata
    metadata = dict(doc.metadata)
    metadata['doc_index'] = idx
    metadata['content_length'] = len(doc.page_content)
    metadatas.append(metadata)

    # Document Page Content
    documents_text.append(doc.page_content)

    # Embedding vetor
    embeddings_list.append(embedding.tolist())

# Add it to the collection
try:
    max_batch = 5000 # batch max is 5461
    for i in range(0, len(embeddings), max_batch):
        self.collection.add(
            embeddings=embeddings_list[i:i + max_batch],
            documents=documents_text[i:i + max_batch],
            ids=ids[i:i + max_batch],
            metadatas=metadatas[i:i + max_batch]
        )
    print(f"Successfully added {len(documents)} documents to the store.")
    print(f"Total documents in collection: {self.collection.count()}")

except Exception as e:
    print(f"Error adding documents to vector store: {e})")
    raise
```

Initialize the Vector Store

```
vector_store = VectorStore()
vector_store
```

```
Vector store insitialized for collection bashdocs
Exisiting documents in collection: 0
```

```
<__main__.VectorStore at 0x7fa303d105c0>
```

Create embeddings

if USE_EXISTING = False then we read in the files from the data directory and add them to the vector store. This will cause duplicates if run multiple times without setting INIT=True to clean the db store.

```
# get the text from the documents and create embeddings to store
if not USE_EXISTING:
    corpus = process_all_files()
    chunks = split_documents(corpus)
    if VERBOSE:
        print(chunks[3])

    texts = [doc.page_content for doc in chunks]

    # Use the EmbeddingMgr class to generate them
    embeddings = emb_mgr.generate_embeddings(texts)

    # Add the documents to the vector store
    vector_store.add_documents(chunks, embeddings)
```

```
Preparing to process 2077 text files...
Total documents loaded: 2077.
<class 'list'>
Splitting 2077 documents into 39773 chunks.

Sample chunk:
Page_content: Reuse and expand the shell history in ‘sh‘, Bash, Zsh, ‘rbash‘, and ‘ksh‘.
  See also: ‘^‘.
  More information: <https://gnu.org/software/bash/manual/bash.html#Event-Designators>.

  Substitute with the previous command and run it with ‘sudo‘:

      sudo !!

  Substitute with a command based on its...
Metadata: {'source': 'manpage', 'manpage': '!.tldr', 'source_file': '!.tldr.txt', 'file_type
```

```
Adding 39773 documents to the vector store...
Successfully added 39773 documents to the store.
Total documents in collection: 39773
```

Prompt and RAG Retrieval Pipeline

Retriever class

```python
class Retriever:
    """Encodes a query and returns similar documents from the vector store db"""

    def __init__(self, vector_store: VectorStore, embedding_manager: EmbeddingMgr):
        """
        ctor

        Args:
            vector_store: The VectorStore of embeddings
            embedding_manager: The EmbeddingMgr that handles encoding embeddings
        """
        self.store = vector_store
        self.manager = embedding_manager

    def fetch(self, query: str, top_k: int = 5, threshold: float = 0.0) -> List[Dict[str, An
        """
        Retrieves documents from the store

        Args:
            query: The user query
            top_k: Number of results to return
            threshold: Minimum similarity score

        Returns:
            List of dictionaries with the documents and metadata
        """
        if VERBOSE:
            print(f"Fetching documents for '{query}'")
            print(f"Top K: {top_k}. Threshold: {threshold}")

        # create the embedding for the query
        query_embedding = self.manager.generate_embeddings([query])[0]

        # Search the vector store
        try:
            results = self.store.collection.query(
                query_embeddings=[query_embedding.tolist()],
                n_results=top_k
```

```
            )

            fetched_docs = []

            if results['documents'] and results['documents'][0]:
                documents = results['documents'][0]
                metadatas = results['metadatas'][0]
                distances = results['distances'][0]
                ids = results['ids'][0]

                for i, (doc_id, document, metadata, distance) in enumerate(zip(ids, document
                    # Convert distance to similarity score - cosine distance for chromadb
                    similarity = 1 - distance

                    if similarity >= threshold:
                        fetched_docs.append({
                            'id': doc_id,
                            'content': document,
                            'metadata': metadata,
                            'similarity_score': similarity,
                            'distance': distance,
                            'rank': i + 1
                        })
                if VERBOSE:
                    print(f"Fetched {len(fetched_docs)} documents (filtered by threshold)")
            else:
                if VERBOSE:
                    print("No documents found")

            return fetched_docs

        except Exception as e:
            print(f"Error fetching documents: {e}")
            return []


# RAG function to pull it all together
def rag_call(query, retriever, llm, top_k=3, pure: bool = False) -> Tuple:
    """Calls the LLM

    Args:
        query: the question asked
        llm: the llm instance to call
        top_k: the number of documents to return for the context
        pure: set to True to call the llm without context for testing
    """
```

```
    # RAG function to pull it all together
    if not pure:
        fetched_docs = retriever.fetch(query, top_k=top_k)
    else:
        fetched_docs = []

    # put all fetched documents into the context
    if fetched_docs:
        found = len(fetched_docs)
        context = "\n\n".join(doc['content'] for doc in fetched_docs)
    else:
        found  = 0
        context = ""

    # Create the prompt
    prompt = f"""You are a proffesional AI assistant theat specializes in asnwering
        questions about Linux commands.
        Use the following context to answer the question accurately and concisely.
        Context:
        {context}

        Question: {query}

        Answer:"""
    try:
        response = llm.invoke([prompt.format(context=context, query=query)])
    except Exception as e:
        print(f"Error:{e}\nContext:{context}")

    return (found, response.content, context)
```

Read the JSON questions and answers

We need to call the function **rag_call(query, retriever, llm, top_k=3)** with each query from the JSON file and capture the LLM answer and the answer from the JSON test answer for each question and compare them. We may be able to use cosine similarity, but a manual inspection will probably be needed for accurate assessment.

submit a single question and return the answer

```
import json
from langchain_groq import ChatGroq

api_key = os.getenv("GROQ_API_KEY")
llm = ChatGroq(groq_api_key=api_key, model_name="llama-3.1-8b-instant", temperature=0.1, max
```

```
fido = Retriever(vector_store, emb_mgr)
```

Load the JSON test data

```
with open("../test/test_questions.json", "r") as jsf:
    test_data = json.load(jsf)
```

```
type(test_data)
```

```
dict
```

```
import pandas as pd
```

```
df_test = pd.DataFrame(test_data)
```

```
df_test.head()
```

```
                                    linux_questions
0  {'id': 1, 'question': 'What is Linux?', 'answe...
1  {'id': 2, 'question': 'What is a Linux distrib...
2  {'id': 3, 'question': 'What is the difference ...
3  {'id': 4, 'question': 'What is the Linux shell...
4  {'id': 5, 'question': 'How do you check the cu...
```

```
results = []
for item in test_data['linux_questions'][:20]:
    result = rag_call(item['question'], fido, llm)
    item['result'] = result
    results.append(item)
```

```
len(results)
```

```
20
```

```
df_results = pd.DataFrame(results)
df_results.head()
```

```
   id                                           question  \
0   1                                      What is Linux?
1   2                       What is a Linux distribution?
2   3  What is the difference between the kernel and ...
3   4                            What is the Linux shell?
4   5    How do you check the current working directory?

                                              answer  \
0  Linux is an open-source, Unix-like operating s...
1  A Linux distribution (distro) bundles the Linu...
2  The kernel manages hardware/resources (CPU, me...
3  The shell is a command interpreter (e.g., bash...
4                                        Use 'pwd'.
```

```
                                                  result
0  (2, Based on the provided context, Linux is no...
1  (0, A Linux distribution, often abbreviated as...
2  (0, In the context of Linux, the kernel and th...
3  (3, The Linux shell is a command-line interpre...
4  (3, To check the current working directory, yo...
```

Function Definitions

Helper to calculate cosine similarity between the provided JSON answer and the LLM answer

```
def calculate_similarity(answer, llm):
    """Calculates the cosine similarity between 2 embeddings"""
    dot_product = np.dot(answer, llm)
    norm_answer = np.linalg.norm(answer)
    norm_llm = np.linalg.norm(llm)
    if norm_answer == 0 or norm_llm == 0:
        return 0
    return dot_product / (norm_answer * norm_llm)
```

Test Loop

```
def test_similarity(data: Dict = None, iterations: int = 20, pure: bool = False, top_k: int
    results = []
    manager = EmbeddingMgr(model_name="all-MiniLM-L6-v2", verbose=False)
    store = VectorStore()
    api_key = os.getenv("GROQ_API_KEY")
    llm = ChatGroq(groq_api_key=api_key, model_name="llama-3.1-8b-instant", temperature=0.1,
    fido = Retriever(store, manager)

    for item in data['linux_questions'][:iterations]:
        result = rag_call(item['question'], retriever=fido, top_k=top_k, llm=llm, pure=pure)
        X = manager.generate_embeddings(item['answer'])
        Y = manager.generate_embeddings(result[1])
        similarity = calculate_similarity(X, Y)
        item['result'] = result
        item['similarity'] = similarity
        item['context'] = result[2]
        results.append(item)


    return results
```

```
answer_emb = emb_mgr.generate_embeddings(df_results['answer'][0])
llm_emb = emb_mgr.generate_embeddings(df_results['result'][0][1])

similarity = calculate_similarity(answer_emb, llm_emb)
```

```
print(similarity)
```

```
0.83712494
```

```
results = test_similarity(test_data, iterations=20, pure=False)
```

```
Loading model all-MiniLM-L6-v2.
```


```
{"model_id":"1e83eef1f49e411ca3a73cc0ad39303e","version_major":2,"version_minor":0}
```


```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you ex
```

```
Model loaded successfully.
Embedding dimension: 384
Vector store insitialized for collection bashdocs
Exisiting documents in collection: 39773
```

```
len(results)
```

```
20
```

Average Similarity (with RAG)

```
df = pd.DataFrame(results)
df.head()
print(df['similarity'].mean())
```

```
0.6739473
```

```
df['similarity'].describe()
```

```
count    20.000000
mean      0.673947
std       0.160474
min       0.368989
25%       0.534039
50%       0.721193
75%       0.795713
max       0.908858
Name: similarity, dtype: float64
```

```
df['similarity'].value_counts()
```

```
similarity
0.650112    1
0.813224    1
```

```
0.857346    1
0.784323    1
0.542530    1
0.434655    1
0.726160    1
0.716226    1
0.770424    1
0.811378    1
0.508566    1
0.368989    1
0.728064    1
0.504971    1
0.790492    1
0.908858    1
0.884839    1
0.635151    1
0.481988    1
0.560650    1
Name: count, dtype: int64
```

Test without RAG

```
results = test_similarity(test_data, iterations=20, pure=False)
```

```
Loading model all-MiniLM-L6-v2.
```

```
{"model_id":"3638c6fa8da6401692b54dc03debdc87","version_major":2,"version_minor":0}
```

```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you ex
```

```
Model loaded successfully.
Embedding dimension: 384
Vector store insitialized for collection bashdocs
Exisiting documents in collection: 39773
```

Average Similarity (without RAG)

```
df = pd.DataFrame(results)
df.head()
print(df['similarity'].mean())
```

```
0.682964
```

```
df['similarity'].describe()
```

```
count    20.000000
mean      0.682964
std       0.157505
min       0.438287
25%       0.512925
50%       0.724904
75%       0.810986
max       0.900163
Name: similarity, dtype: float64
```

```
df['similarity'].value_counts()
```

```
similarity
0.807866    1
0.820344    1
0.857161    1
0.769631    1
0.514378    1
0.446423    1
0.733581    1
0.716226    1
0.785968    1
0.833099    1
0.508566    1
0.438287    1
0.700565    1
0.485085    1
0.762539    1
0.900163    1
0.884839    1
0.635151    1
0.481988    1
0.577421    1
Name: count, dtype: int64
```

Test with RAG and varying top_k

```
results = test_similarity(test_data, top_k=2, iterations=20, pure=False)
```

```
Loading model all-MiniLM-L6-v2.
```

{"model_id":"14b0435313c740e588bf6fed16bc7d92","version_major":2,"version_minor":0}

```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |
```

```
Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you ex
```

```
Model loaded successfully.
Embedding dimension: 384
Vector store insitialized for collection bashdocs
Exisiting documents in collection: 39773
```

```
df = pd.DataFrame(results)
df.head()
print(df['similarity'].mean())
```

```
0.7003068
```

```
df['similarity'].describe()
```

```
count    20.000000
mean      0.700307
std       0.165748
min       0.441106
25%       0.553128
50%       0.766734
75%       0.825781
max       0.946316
Name: similarity, dtype: float64
```

```
df['similarity'].value_counts()
```

```
similarity
0.946316    1
0.832863    1
0.870156    1
0.787739    1
0.574886    1
0.441106    1
0.793848    1
0.753346    1
0.782314    1
0.823420    1
0.495276    1
0.481128    1
0.706194    1
0.503737    1
0.780122    1
0.908318    1
0.884839    1
0.595272    1
0.475661    1
```

```
0.569591    1
Name: count, dtype: int64
```

```
results = test_similarity(test_data, iterations=len(test_data['linux_questions']), pure=Fals
```

```
Loading model all-MiniLM-L6-v2.
```

{"model_id":"d039b684f8fd43c4a2e7c7c31efa1368","version_major":2,"version_minor":0}

```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you ex
```

```
Model loaded successfully.
Embedding dimension: 384
Vector store insitialized for collection bashdocs
Exisiting documents in collection: 39773
```

```
len(results)
```

```
50
```

```
df = pd.DataFrame(results)
df.head()
print(df['similarity'].mean())
```

```
0.7058017
```

```
df['similarity'].describe()
```

```
count    50.000000
mean      0.705802
std       0.144642
min       0.379336
25%       0.601265
50%       0.735767
75%       0.823115
max       0.946354
Name: similarity, dtype: float64
```

```
df['similarity'].value_counts()
```

```
similarity
0.946354    1
0.813224    1
0.865219    1
0.779563    1
0.520586    1
```

```
0.435932    1
0.725773    1
0.716226    1
0.763754    1
0.816742    1
0.508566    1
0.379336    1
0.763295    1
0.483630    1
0.791351    1
0.908332    1
0.884839    1
0.635151    1
0.481988    1
0.560650    1
0.768934    1
0.846633    1
0.762297    1
0.853005    1
0.609912    1
0.427351    1
0.825239    1
0.601240    1
0.587672    1
0.851522    1
0.674750    1
0.873378    1
0.744418    1
0.585009    1
0.537275    1
0.662817    1
0.756639    1
0.622612    1
0.601341    1
0.780249    1
0.660845    1
0.644629    1
0.599699    1
0.771834    1
0.624147    1
0.857302    1
0.853073    1
0.928606    1
0.870031    1
0.727116    1
Name: count, dtype: int64
```

```
VERBOSE = True
```

```
fido.fetch("how to use the file command")
```

```
Fetching documents for 'how to use the file command'
Top K: 5. Threshold: 0.0
Generating embeddings for 1 strings.
Generated embeddings with shape: (1, 384)
Fetched 5 documents (filtered by threshold)
```


```
[{'id': 'doc_c63fde82_18474',
  'content': 'Use of the -a argument is highly recommended with this option.\n\nThe map_file
  'metadata': {'doc_index': 18474,
   'platform': 'linux',
   'file_type': 'txt',
   'source': 'manpage',
   'manpage': 'gprof',
   'source_file': 'gprof.txt',
   'content_length': 233},
  'similarity_score': 0.18758410215377808,
  'distance': 0.8124158978462219,
  'rank': 1},
 {'id': 'doc_31bd400f_21763',
  'content': 'files, they are all inserted into the list of files and the first one is\nexam
  'metadata': {'source_file': 'less.txt',
   'manpage': 'less',
   'file_type': 'txt',
   'platform': 'linux',
   'content_length': 453,
   'source': 'manpage',
   'doc_index': 21763},
  'similarity_score': 0.17342209815979004,
  'distance': 0.82657790184021,
  'rank': 2},
 {'id': 'doc_17cd38d2_22028',
  'content': '#command\n\nIf the command section is the first section in the file, this line
  'metadata': {'manpage': 'lesskey',
   'platform': 'linux',
   'source': 'manpage',
   'content_length': 215,
   'source_file': 'lesskey.txt',
   'file_type': 'txt',
   'doc_index': 22028},
  'similarity_score': 0.17287170886993408,
  'distance': 0.8271282911300659,
  'rank': 3},
 {'id': 'doc_85a2c806_404',
```

```
  'content': 'give as a command line that has zero or more input file names.\tThe input\nfil
  'metadata': {'source': 'manpage',
   'platform': 'linux',
   'manpage': 'as',
   'source_file': 'as.txt',
   'file_type': 'txt',
   'doc_index': 404,
   'content_length': 449},
  'similarity_score': 0.11734127998352051,
  'distance': 0.8826587200164795,
  'rank': 4},
 {'id': 'doc_77273d03_8327',
  'content': "arguments where it is alone, as in some versions of Both of these\nconstructio
  'metadata': {'platform': 'linux',
   'source': 'manpage',
   'file_type': 'txt',
   'manpage': 'find',
   'doc_index': 8327,
   'source_file': 'find.txt',
   'content_length': 444},
  'similarity_score': 0.11491656303405762,
  'distance': 0.8850834369659424,
  'rank': 5}]
```


Citations

IBM Technology LangChain vs LangGraph: A Tale of Two Frameworks https://youtu.be/qAF1NjEVHhY?si=CKShLFYuOz0xbi1N

Krish Naik Complete RAG Crash Course With Langchain In 2 Hours https://youtu.be/o126p1QN_RI?si=7xC5H47A3iuu52RK

pixegami Python RAG Tutorial (with Local LLMs): AI For Your PDFs https://youtu.be/2TJxpyO3ei4?si=zPNEsAFWWy5Cenzq

Technologies, Cuantum. Natural Language Processing with Python Updated Edition: From Basics to Advanced Projects: Mastering Text Analysis, Machine Learning Models, and Chatbot ... (Mastering the AI Revolution Book 4) (p. 1). (Function). Kindle Edition.

Tunstall, Lewis; Werra, Leandro von; Wolf, Thomas. Natural Language Processing with Transformers, Revised Edition (p. 4). (Function). Kindle Edition.